

Reglas de trabajo

Sistema RQ · Grupo Copacabana · agosto de 2026

Este documento recoge lo que se aprendió construyendo el sistema de requerimientos, sobre todo de los errores. Casi todas las reglas nacieron de un fallo concreto que llegó a producción o estuvo a punto: por eso cada una lleva el porqué. Sirven para este proyecto y para los siguientes.

Cuatro palabras para pedir verificación

No hace falta explicar qué hacer cada vez. Basta con decir una de estas.

Se dice	Qué ocurre	Tarda	Cuándo
atácalo	Varios revisores en paralelo intentan romper lo recién construido, cada uno desde un ángulo distinto	10–20 min	Antes de correr una migración. Siempre que se toque dinero o permisos
verifica la base	Consultas que comprueban que lo que hay en producción coincide con el repositorio	1 min	Después de correr cualquier migración, sin excepción
corre las pruebas	Las pruebas automáticas de los cálculos	segundos	Después de tocar cualquier cálculo
pásame la lista	La lista de pruebas manuales, ordenada por perfil	—	Antes de dar algo por terminado

El más barato es el que más veces ha salvado el proyecto. "Verifica la base" detectó una migración que había fallado a medias sin avisar, y otro día que dos parches no estaban aplicados aunque otra comprobación decía que sí.

Al construir

1 · No reescribir nada sin repasar antes qué hacía.

Leer la versión anterior línea por línea, enumerar todo lo que hacía, y tacharlo contra la nueva. Lo que se quite, se quita a propósito y se dice. Pasó dos veces en una semana: al reescribir una regla se perdió el sello de "quién compró", y el comprador se quedó sin poder facturar en producción sin que nadie lo notara.

2 · Toda protección necesita su gemela en el alta.

Una regla que solo vigila las modificaciones no protege nada: basta con crear el registro ya en el estado prohibido. Se descubrió tres veces seguidas — salidas que nacían aprobadas, préstamos que nacían activos moviendo inventario, y cajas que nacían con el fondo falseado.

3 · Las firmas las pone el servidor, nunca el navegador.

Quién aprobó, quién anuló, quién pagó, quién corrigió. Un dato que el cliente puede escribir no es una firma: es una sugerencia. Sin esto, una persona podía anular material firmando con el nombre de otra.

4 · Nada se edita en silencio.

Todo cambio que altere cantidades, dinero o decisiones lleva motivo obligatorio, y el sistema añade quién y cuándo. Si algo se puede corregir sin explicar por qué, se corregirá sin que nadie se entere.

5 · Para saber de qué día es una hora guardada, hay que convertirla.

Las bases guardan en hora universal y Perú va cinco horas por detrás: a partir de las 19:00, lo guardado ya lleva la fecha del día siguiente. Leerle el prefijo al texto funciona toda la mañana y falla todas las tardes — que suele ser justo cuando se cierra la jornada.

6 · La migración va antes que el código que la usa.

Desplegar código que espera una columna que aún no existe deja el sistema caído para todos. Y toda migración se guarda en el repositorio antes de ejecutarla, nunca solo por chat: si no está escrita, no existe para quien tenga que reconstruir la base.

Al diseñar pantallas

7 · Lo pendiente ocupa la pantalla; lo hecho se consulta.

Toda lista de cosas terminadas solo crece, y acaba enterrando lo que sí requiere atención. Con 210 facturas pagadas encima, las que había que pagar quedaban abajo. Se archivan a un clic, nunca se borran, y el mensaje cuando no queda nada es "todo al día", que no es lo mismo que "sin datos".

8 · Una alerta que salta siempre deja de leerse.

Cuarenta avisos idénticos no son cuarenta problemas: son un problema mal contado. Se agrupan, y quien manda puede darlos por resueltos con un motivo. Si no, la alerta que sí importa queda enterrada entre las viejas.

9 · Un botón que no puede cumplir es peor que no tenerlo.

Si el sistema va a rechazar la acción, el botón no debe estar. Y si algo no se puede hacer, hay que decir por qué y qué hacer en su lugar — no dejar el botón gris sin explicación.

10 · El mensaje de error lo lee alguien en obra, no un programador.

Nada de textos técnicos en inglés. Explicar qué pasó, por qué, y qué hacer ahora.

Sobre probar

11 · Lo que está "pendiente de probar" está roto.

No "probablemente bien": roto, hasta que una persona lo use y funcione. De los problemas graves encontrados en una semana, tres llevaban días rotos y los tres estaban apuntados como pendientes de verificar. La lista de pendientes de probar es el riesgo real de un lanzamiento, más que la de pendientes de construir.

12 · Mirar la pantalla encuentra lo que ninguna revisión de código encuentra.

Los fallos de modelo — que el sistema haga bien algo que no es lo que pasa en la realidad — solo aparecen usándolo. Media hora de alguien que conoce el negocio vale más que cualquier análisis: así se descubrió que la caja chica no era un fondo fijo y que había dinero entregado sin sitio donde constar que volvió.

13 · Un verificador que da falsos positivos es peor que no tenerlo.

Si avisa de algo que está bien, se deja de mirar. Y si comprueba lo fácil pero no lo que importa, da tranquilidad falsa: hubo un verificador que decía 47 de 47 correctos sin comprobar ni una sola regla de permisos.

14 · Refactorizar sin pruebas no es refactorizar: es reescribir y confiar.

Un refactor se define como un cambio que no altera el comportamiento. Sin forma de comprobar el comportamiento, esa afirmación no la puede hacer nadie. Primero las pruebas, después el reordenamiento.

El método que funcionó: construir → atacar → verificar la base → probarlo en pantalla con alguien que conozca el negocio. Saltarse el último paso fue lo que dejó tres cosas rotas durante días.